The **Missionaries and Cannibals Problem** is a famous problem used in **Artificial Intelligence (AI)** to explain **state-space search and problem solving**.

Problem Description

Three **missionaries** and three **cannibals** are on one side of a river and want to cross to the other side using a **boat**.

Conditions:

1. The boat can carry **at most 2 people** at a time.
2. At least **one person must row the boat**.
3. If **cannibals outnumber missionaries on either side**, the missionaries will be eaten.
4. The goal is to move **all 3 missionaries and 3 cannibals safely to the other side**.

State Representation

A state is represented as:

(M_left, C_left, Boat)

- **M_left** = number of missionaries on the left bank
- **C_left** = number of cannibals on the left bank
- **Boat** = position of the boat (Left / Right)

Initial State:
(3, 3, Left)

Goal State:
(0, 0, Right)

Possible Actions

The boat can carry these combinations:

- 1 Missionary (1M)
- 2 Missionaries (2M)
- 1 Cannibal (1C)
- 2 Cannibals (2C)
- 1 Missionary + 1 Cannibal (1M,1C)

But only **valid states** (where missionaries are not outnumbered) are allowed.

Left Bank	Right Bank	Safe?
(3M,3C)	(0,0)	✓ Safe
(2M,2C)	(1M,1C)	✓ Safe
(3M,1C)	(0,2C)	✓ Safe
(0M,2C)	(3M,1C)	✓ Safe (no missionaries to be eaten)

Left Bank	Right Bank	Reason
(1M,3C)	(2M,0C)	Left side cannibals > missionaries ❌
(2M,3C)	(1M,0C)	Left side missionaries outnumbered ❌

Missionaries should never be fewer than cannibals on any side

One valid Solution

Step	Move	Left Bank (M,C)	Right Bank (M,C)
Start	Initial	(3,3)	(0,0)
1	2 Cannibals cross	(3,1)	(0,2)
2	1 Cannibal returns	(3,2)	(0,1)
3	2 Cannibals cross	(3,0)	(0,3)
4	1 Cannibal returns	(3,1)	(0,2)
5	2 Missionaries cross	(1,1)	(2,2)
6	1M + 1C return	(2,2)	(1,1)
7	2 Missionaries cross	(0,2)	(3,1)
8	1 Cannibal returns	(0,3)	(3,0)
9	2 Cannibals cross	(0,1)	(3,2)
10	1 Missionary returns	(1,1)	(2,2)
11	1M + 1C cross	(0,0)	(3,3)

CODE

```
from collections import deque


# Check if state is valid
def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and c > m:
        return False
    if (3-m) > 0 and (3-c) > (3-m):
        return False
    return True


def bfs():
    start = (3,3,'L')
    goal = (0,0,'R')


    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    queue = deque()
    queue.append((start, []))
    visited = set()


    while queue:
        (m,c,boat), path = queue.popleft()


        if (m,c,boat) == goal:
            return path + [(m,c,boat)]
```

```
        if (m,c,boat) in visited:
            continue

        visited.add((m,c,boat))

        for dm,dc in moves:
            if boat == 'L':
                new = (m-dm, c-dc, 'R')
            else:
                new = (m+dm, c+dc, 'L')

            nm,nc,nb = new

            if valid(nm,nc):
                queue.append((new, path + [(m,c,boat)]))

solution = bfs()

print("\nSolution Table\n")
print("| Step | Move | Left Bank (M,C) | Right Bank (M,C) |")
print("|------|------|-----------------|------------------|")

prev = solution[0]
print(f"| Start | Initial | ({prev[0]},{prev[1]}) | ({3-prev[0]},{3-prev[1]}) |")

step = 1
```

```
for state in solution[1:]:
    m,c,b = state
    pm,pc,pb = prev

    dm = abs(pm-m)
    dc = abs(pc-c)

    if pb == 'L':
        direction = "cross"
    else:
        direction = "return"

    move = ""
    if dm==2: move = "2 Missionaries"
    elif dc==2: move = "2 Cannibals"
    elif dm==1 and dc==1: move = "1M + 1C"
    elif dm==1: move = "1 Missionary"
    elif dc==1: move = "1 Cannibal"

    print(f"| {step} | {move} {direction} | ({m},{c}) | ({3-m},{3-c}) |")

    prev = state
    step += 1
```

Output

Solution Table

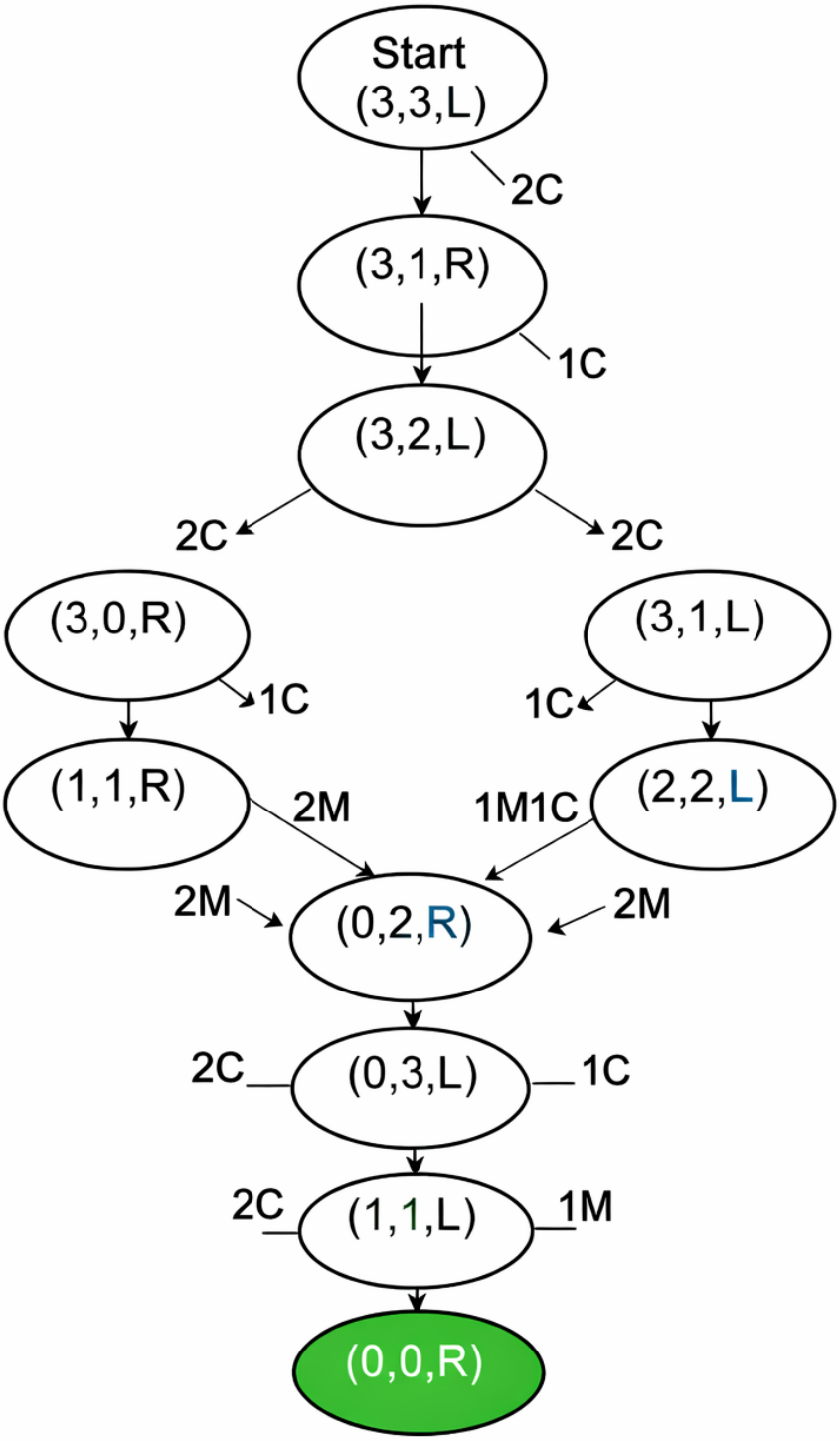
Step	Move	Left Bank (M,C)	Right Bank (M,C)
Start	Initial	(3,3)	(0,0)
1	2 Cannibals cross	(3,1)	(0,2)
2	1 Cannibal return	(3,2)	(0,1)
3	2 Cannibals cross	(3,0)	(0,3)
4	1 Cannibal return	(3,1)	(0,2)
5	2 Missionaries cross	(1,1)	(2,2)
6	1 Missionary + 1 Cannibal return	(2,2)	(1,1)
7	2 Missionaries cross	(0,2)	(3,1)
8	1 Cannibal return	(0,3)	(3,0)
9	2 Cannibals cross	(0,1)	(3,2)
10	1 Missionary return	(1,1)	(2,2)
11	1 Missionary + 1 Cannibal cross	(0,0)	(3,3)

✅ **Goal State Reached:**
All **3 missionaries and 3 cannibals** safely move to the **right bank**.

This is the **optimal solution found using Breadth-First Search**, which explores states level-by-level for the **Missionaries and Cannibals Problem**.

Missionaries and Cannibals Problem

State Space Diagram (BFS)



Program Trace

1. Import BFS Queue

from collections import deque

Explanation

- deque is a **double-ended queue**.
- It is used to implement **Breadth-First Search**.

Example queue operation:

```
queue = deque()
queue.append(state)
queue.popleft()
```

BFS works in **FIFO (First In First Out)** order.

---

2. Valid State Function

def valid(m, c):

Here:

- m = missionaries on **left bank**
- c = cannibals on **left bank**

Example state:

(3,3)

Condition 1

```
if m < 0 or c < 0 or m > 3 or c > 3:
    return False
```

This prevents **invalid numbers**.

Example invalid states:

(-1,2) ❌
(4,1) ❌

Condition 2

```
if m > 0 and c > m:
    return False
```

Meaning:

Cannibals > Missionaries

Example:

(1M,3C) ❌ missionaries eaten

Condition 3

```
if (3-m) > 0 and (3-c) > (3-m):
    return False
```

This checks the **right bank**.

Example:

If left side:

(1M,2C)

Right side becomes:

(2M,1C)

Safe.

But if right side:

(1M,3C)

Missionaries eaten ❌

Return Valid

```
return True
```

Meaning state is **safe**.

---

3. BFS Function

```
def bfs():
```

This function finds the **solution path**.

---

4. Initial State

start = (3,3,'L')

Meaning:

Left side : 3M 3C
Boat : Left

Representation:

(3,3,L)

---

5. Goal State

goal = (0,0,'R')

Meaning:

Left side : 0M 0C
Right side : 3M 3C
Boat : Right

---

6. Possible Boat Moves

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

Meaning boat can carry:

Move	Meaning
(1,0)	1 Missionary
(2,0)	2 Missionaries
(0,1)	1 Cannibal
(0,2)	2 Cannibals
(1,1)	1M + 1C

---

7. Create Queue

queue = deque()
queue.append((start, []))

Queue contains:

[((3,3,L) , [])]

Second part [] stores **path history**.

---

8. Visited Set

visited = set()

Stores states already explored.

Example:

visited = {(3,3,L),(3,1,R)}

---

9. BFS Loop

while queue:

Loop runs **until queue becomes empty**.

---

10. Remove First State

(m,c,boat), path = queue.popleft()

Example first iteration:

(m,c,boat) = (3,3,L)
path = []

---

11. Check Goal

if (m,c,boat) == goal:

Check if:

(0,0,R)

If yes → solution found.

---

12. Skip Visited

```
if (m,c,boat) in visited:
    continue
```

Avoid repeating states.

---

13. Mark State Visited

```
visited.add((m,c,boat))
```

Now:

visited = {(3,3,L)}

---

14. Try All Boat Moves

```
for dm,dc in moves:
```

Example iteration:

dm=1 dc=0
dm=2 dc=0
dm=0 dc=1
dm=0 dc=2
dm=1 dc=1

---

15. Boat on Left

```
if boat == 'L':
    new = (m-dm, c-dc, 'R')
```

Example:

(3,3,L) + move (0,2)

Result:

(3,1,R)

Meaning:

2 Cannibals cross

---

16. Boat on Right

else:
    new = (m+dm, c+dc, 'L')

Example:

(3,1,R)

Return 1 cannibal:

(3,2,L)

---

17. Extract New State

nm,nc,nb = new

Example:

nm = 3
nc = 1
nb = R

---

18. Check Valid State

if valid(nm,nc):

Example:

(3,1) ✓ safe

---

19. Add to Queue

queue.append((new, path + [(m,c,boat)]))

Queue becomes:

[(3,1,R)]

Path stored:

[(3,3,L)]

---

20. Continue BFS

Queue expands like:

(3,3,L)
(3,1,R)
(3,2,L)
(3,0,R)
(3,1,L)
(1,1,R)
(2,2,L)
(0,2,R)
(0,3,L)
(0,1,R)
(1,1,L)
(0,0,R)

---

21. Goal Found

Final state:

(0,0,R)

Meaning:

Left side : 0M 0C
Right side : 3M 3C
Boat : Right

Final Solution Path

(3,3,L)
(3,1,R)
(3,2,L)
(3,0,R)
(3,1,L)
(1,1,R)
(2,2,L)
(0,2,R)
(0,3,L)
(0,1,R)
(1,1,L)
(0,0,R)